

HeapsPermutation.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.Arrays;
import java.util.stream.IntStream;

public class HeapsPermutation {
    public static void swap(int[] A, int i, int j) {
        A[i] = (A[i] + A[j]) - (A[j] = A[i]);
    }
    static List allPermutations(Integer n) {
        int[] A = IntStream.rangeClosed(0, n-1).toArray();
        List Permutations = new ArrayList<ArrayList<Integer>>();
        int[] C = new int[n];
        // initialize stack encoding recursion/position of next transposition
        Arrays.fill(C, 0);
        // store initial permutation as List
        // in typical complicated Java way
        Permutations.add(Arrays.stream(A).boxed().toList());
        int i = 1;
        while(i < n) {
            if(C[i] < i) {
                if(i%2 == 0)
                    swap(A, 0, i);
                else
                    swap(A, C[i], i);
                Permutations.add(Arrays.stream(A).boxed().toList());
                C[i] += 1;
                i = 1;
            }
            else {
                C[i] = 0;
                i += 1;
            }
        }
        return Permutations;
    }
}
```